NDI Metadata

NDI includes the ability to send XML formatted metadata messages between NDI senders and receivers using several different methods. This is a powerful mechanism that can be used to communicate status, provide for remote control, and many other purposes. In particular, the ability to send messages bidirectionally between NDI senders and receivers allows NDI to simplify workflows that would be significantly more complex if implemented with other audio/video streaming protocols.

Metadata Sources

Metadata (NDIlib_metadata_frame_t)

This is the "normal" metadata channel for sending messages between NDI senders and receivers. This channel is bidirectional and is used for generic communications which do not have a specific association with a particular audio or video frame.

Connection Metadata

Connection metadata is a special class of metadata frames that can be registered with the NDI SDK for both the sender and receiver. Upon establishing a new NDI connection, the connection metadata is sent from sender to receiver and from receiver to sender. There are several standard connection metadata elements including `<ndi_product>`, `<ndi_capabilities>`, and `<ndi_format>`.

Connection metadata frames are sent automatically to each existing and new connection after being registerd with the NDI SDK and are received as normal metadata frames.

Audio Frame Metadata

Audio frame metadata is used for communicating information that is specific to a particular audio frame. There are currently no specific metadata elements defined for audio metadata frames.

Video Frame Metadata

Video frame metadata is used for communicating information that is specific to a particular video frame. Example metadata elements that should be passed as video frame metadata include `<ndi_tracking_info>`, `<ndi_color_info>`, and `<vancData>`.

NDI Metadata and XML

While NDI metadata is formatted as XML, there are some differences between XML used for NDI metadata frames and a more conventional stand-alone XML file. This section discusses how XML standards apply to NDI metadata frames.

XML Prolog

NDI metadata frames should not contain an XML prolog. Each NDI Metadata frame can be assumed to have the following prolog, indicating XML version 1.0 compliance and UTF-8 encoding.

```
<?xml version="1.0" encoding="UTF-8"?>
```

XML Syntax

NDI metadata frames should be "Well Formed" XML documents with correct syntax:

- Metadata should have one and only one root element
- XML elements must have a closing tag
- XML tags are case sensitive
- XML elements must be properly nested
- XML attribute values must be quoted

XML Root Element

As a "Well Formed" XML document, an NDI metadata frame can only have one root element. If there is a need to pass more than one XML element in an NDI metadata frame, the root element should be set to `<ndi_metadata_group>` and the required XML elements can be passed as children to this root element. See the documentation for the `<ndi_metadata_group>` element, below, for example usage.

XML Comments

XML comments are allowed but discouraged in NDI metadata frames.

XML Namespaces

XML namespaces are not supported in NDI metadata frames.

XML Element Names

With the exception of element names defined in this document, no XML element passed in a metadata frame to the NDI SDK should begin with the string `ndi_` or `ntk_` (or any permutation of capitalization,eg: `NDi_`).

Validation

Schema Files

Example XML files and XSD files are provided allowing standard XML validation tools to be used to validate NDI metadata messages. A parser which understands XSD version 1.1 is required. This can be as simple as something like the following python code (tested on Debian Bookworm with the python3-xmlschema package installed):

```
$ python3 -q
>>> import xmlschema
>>> schema = xmlschema.XMLSchema11('Schemas/ndi_metadata_all.xsd')
>>> schema.is_valid('Documents/ndi_color_info.xml')
True
>>> schema.is_valid('Documents/vancData.Multiple.xml')
True
>>> exit()
```

Several top-level schema files are provided to assist with validating the various metadata streams available.

- `ndi_metadata_all` : A collection of all valid NDI metadata messages, regardless of how or where they are sent
- `ndi_metadata_recv` : Valid metadata messages received by an NDI receiver
- `ndi_metadata_send` : Valid metadata messages received by an NDI sender
- `ndi_metadata_video` : Valid metadata messages received with an NDI video frame

The remaining file in the `Schemas` directory define specific metadata elements and are pulled in by the top-level files listed above.

There is a Liquid Studio project file `NDI_Metadata.lxsproj` in the top level directory which can be used with the community (or paid) version of Liquid Studio to assist with editing and validation of the NDI metadata and schema files.

Schema Limitations

There are a few known limitations to the schema files:

- The logic for the use of the various attributes available under the `ndi_capabilities` element is currently (2025.02.03) incomplete and under review.
- No user defined element name should ever match the regex `[nN][dD][iI].*` or `[nN][tT][kK].*`, which is not currently expressed in the schema files.
- There are currently no specific metadata elements defined for sending with an NDI audio frame.

XML Files

Typical examples of various NDI metadata frames are provided in the `Documents` directory.

Validation Application

An application is currently (2025.02.03) in development that will listen for metadata from an NDI sender or receiver and can be used to validate the formatting and in some cases the content of an NDI metadata frame.

Metadata Elements

`<ndi_product>` Element

- Initial Implementation: NewTek
- Location: Connection Metadata

Used by both NDI senders and receivers to indicate product details.

```
<ndi_product
    long_name="NDILib Receive Example."
    short_name="NDILib Receive"
    manufacturer="CoolCo, Inc."
    model_name="PBX-42Q"
    version="1.000.000"
    serial="ABCDEFG"
    session_name="My Midday Show" />
```

`<ndi_product>` Attributes

- long_name
  - Full human readable product name
- short_name
  - Abbreviated human readable product name
- manufacturer
  - Product manufacturer
- version
  - Product firmware version
- model_name
  - Product model name
- serial
  - Product serial number
- session_name
  - Session name for TriCaster or similar products
  - General product information for other devices

`<ndi_capabilities>` Element

- Initial Implementation: NewTek
- Location: Connection Metadata

Indicated product capabilities for both NDI senders and receivers. Most capabilities are sender specific (ptz and exposure control) but NDI receivers will often have a web_control URL.

```
<ndi_capabilities
    web_control="http://ndi.video/"
    ntk_ptz="true"
    ntk_exposure_v2="true" />
```

`<ndi_capabilities>` Attributes

- web_control=“URL”

  - The URL to the local device webpage. If %IP% is present in the value, it will be replaced with the local IP of the NIC in which the NDI receiver is connected to.
- ntk_ptz="true"
  - Signifies that this NDI sender is capable of processing PTZ commands sent from the NDI receiver. The NDI receiver will only assume the NDI sender can support PTZ commands if this attribute is received and set to the value "true".
- ntk_pan_tilt="true"
  - The NDI sender supports pan and tilt control.
- ntk_zoom="true"
  - The NDI sender supports zoom control.
- ntk_iris="true"
  - The NDI sender supports iris control.
- ntk_white_balance="true"
  - The NDI sender supports white balance control.
- ntk_exposure="true"
  - The NDI sender supports exposure control.
- ntk_exposure_v2="true"
  - The NDI sender supports detailed control over exposure such as iris, gain, and shutter speed.
- ntk_focus="true"
  - The NDI sender supports manual focus control.
- ntk_autofocus="true"
  - The NDI sender supports setting auto focus.
- ntk_preset_speed="true"
  - The NDI sender has preset speed support.

`<ndi_format>` Element

- Initial Implementation: NewTek
- Location: Connection Metadata

Sent by an NDI receiver to indicate it's preferred or native format.

```
<ndi_format>
  <video_format xres="1920" yres="1080"
                frame_rate_n="60000" frame_rate_d="1001"
                aspect_ratio="1.77778" progressive="true"
  />
  <audio_format no_channels="4" sample_rate="48000"/>
</ndi_format>
```

`<ndi_format>` Children

- `<video_format>`
- `<audio_format>`

`<video_format>` Attributes

- xres
  - Horizontal resolution in pixels
- yres
  - Vertical resolution in lines
- frame_rate_n
  - Numerator portion of the frame rate
- frame_rate_d
  - Denominator portion of the frame rate
- aspect_ratio
  - The picture aspect ratio, 0 means square pixel
- progressive
  - Indicates progressive format when "true" or interlaced format when "false"

<audio_format> Attributes

- no_channels
  - The number of audio channels
- sample_rate
  - The number of audio samples per second

<ndi_color_info> Element

- Initial Implementation: Vizrt
- Location: Video Frame Metadata
- Initial NDI Version: 6.0

The <ndi_color_info> element provides colorimitry details for the associated video frame.

```
<ndi_color_info
    transfer="bt_2100_hlg"
    matrix="bt_2020"
    primaries="bt_2020"
/>
```

<ndi_color_info> Attributes

- primaries
  - Specifies the chromaticity coordinates of the color primaries of the video frame.
  - "bt_601", "bt_709", "bt_2020", or "bt_2100"
- transfer
  - Specifies the brightness transfer characteristic.
  - "bt_601", "bt_709", "bt_2020", "bt_2100_hlg", or "bt_2100_pq"
- matrix
  - Specifies the matrix coefficients used in deriving luma and chroma from RGB or XYZ primaries.
  - "bt_601", "bt_709", "bt_2020", or "bt_2100"

<ndi_metadata_group> Element

- Initial Implementation: NewTek
- Location: Metadata (all flavors)
- Initial NDI Version: 6.0 for video frame metadata, 6.1 for generic metadata frames

Properly formatted XML allows only one root element. If multiple metadata elements need to be attached to a single audio or video frame, they should be grouped together as child elements of an ndi_metadata_group root element.

```
<ndi_metadata_group>
  <ndi_color_info>
    <!-- ndi_color_info content goes here -->
  </ndi_color_info>
  <ndi_tracking_info>
    <!-- ndi_tracking_info content goes here -->
  </ndi_tracking_info>
  <!-- Additional element tags can go here -->
</ndi_metadata_group>
```

<ndi_tracking_info> Element

- Initial Implementation: Vizrt
- Location: Video Frame Metadata
- Initial NDI Version: 5.x

```
<ndi_tracking_info version="1.0.0">
    <package type="binary" protocol="FreeD">
        <!-- Timestamp is optional -->
```

```
        <!-- The tag name is optional, if not present we mean capture timestamp -->
        <timestamp
            name="capture"
            type="xs:dateTimeStamp">2022-07-25T13:20:00.123Z
        </timestamp>
        <!-- We expect binary.base64 string here -->
        <data>MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=</data>
    </package>

    <package type="axis" protocol="FreeD">
        <!-- Timestamp is optional -->
        <timestamp
            name="capture"
            type="xs:dateTimeStamp">2004-04-12T13:20:00.123-05:00
        </timestamp>
        <!-- the naming must be present and unique for the package -->
        <!-- The datatype is mandatory! -->
        <axis name="id" type="xs:integer" value="1" />
        <axis name="posx" type="xs:integer" value="237" />
        <axis name="posy" type="xs:integer" value="9356" />
        <axis name="posz" type="xs:integer" value="44" />
        <axis name="rotx" type="xs:integer" value="23" />
        <axis name="roty" type="xs:integer" value="34" />```
        <axis name="rotz" type="xs:integer" value="43" />
        <axis name="zoom" type="xs:integer" value="12" />
        <axis name="focus" type="xs:integer" value="6785" />
        <axis name="iris" type="xs:integer" value="12" />
        <axis name="extender" type="xs:boolean" value="true" />
    </package>

    <package type="axis" protocol="TrackMen">
        <timestamp
            name="capture"
            type="xs:dateTimeStamp">2004-04-12T13:20:00.123Z
        </timestamp>
        <axis name="posx" type="xs:float" value="23.3" />
        <axis name="posy" type="xs:float" value="45.6" />
        <axis name="posz" type="xs:float" value="44.12223" />
        <axis name="rotx" type="xs:float" value="34" />
        <axis name="roty" type="xs:float" value="45" />
        <axis name="rotz" type="xs:float" value="56.006" />
        <axis name="zoom" type="xs:integer" value="23" />
        <axis name="focus" type="xs:integer" value="7877" />
        <axis name="k1" type="xs:float" value="0.123" />
        <axis name="k2" type="xs:float" value="1.4" />
        <axis name="chipX" type="xs:float" value="23" />
        <axis name="chipY" type="xs:float" value="34" />
        <axis name="centerX" type="xs:float" value="233" />
        <axis name="centerY" type="xs:float" value="0.45" />
    </package>
</ndi_tracking_info>
```

`<ndi_tracking_info>` Attributes

- version
  - The `<ndi_tracking_info>` element must have a version attribute
  - The version attribute is defined as string with the regex format: `[0-9]+.[0- 9]+.[0-9]+`

<ndi_tracking_info> Children

- <package>
  - <ndi_tracking_info> can contain zero or more <package> elements

<package> Element The <package> element holds one single tracking package. This element contains no text.

```
<package
    type="binary"
    protocol="name of protocol">
    <!-- optional --> <timestamp...>
    <data...>
</package>
```

or

```
<package
    type="axis"
    protocol="name of protocol">
    <!-- optional --> <timestamp.../>
    <axis.../>
</package>
```

<package> Attributes

- type
  - The <package> element must have an attribute named type with a value of either `binary` or `axis`
- protocol
  - The value of the attribute `protocol` is the name of the tracking protocol which is embedded in the XML description of <package>
  - This attribute is mandatory
  -

<package> Children

- <timestamp>
  - The <timestamp> element is optional
- <data>
  - If the type of the is binary then exactly one <data> element is mandatory.
- <axis>
  - If the type of the is axis then one or more <axis> elements must be present.

<timestamp> Element When present, the element can provide some additional timestamp information different from the timestamp on the NDI video frame. This element contains no text and has no children.

```
<timestamp
  name="type of timestamp"
  type="xs:dateTimeStamp">13:20:00.123Z
</timestamp>
```

<timestamp> Attributes

- name
  - Is optional and defines the process step when the timestamp was taken. If name is not given, the timestamp will be marked as capture which means the time the video frame was recorded.
  - If the name attribute is not capture, then the timestamp on the NDI video frame is taken as capture timestamp.
  - Please see Appendix Timestamp names.
- type

  - Is mandatory and has a fixed value of xs:dateTimeStamp. The timestamp value must follow the xs:dateTimeStamp format. Please see https://www.w3.org/TR/xmlschema11-2/#dateTimeStamp. A UTC timestamp is preferrable.

<data> Element The element contains the binary representation of the protocol. This element has no attributes and no children. The text value must be encoded in the xs:base64Binary format. Please see https://www.w3schools.com/xml/schema_dtypes_misc.asp

```
<!-- We expect binary.base64 string here -->
<data> MTIzNDU2Nzg5MDEyMzQ1Njc4OTA=</data>
```

<axis> Element The element contains data for one axis. This element contains no text and has no children.

```
<!-- name and type are mandatory! -->
<axis name="id" type="xs:integer" value="1" />
<axis name="posx" type="xs:integer" value="23" />
```

<axis> Attributes

- name
  - Identifies the axis. If the name is known by the Tracking Hub, the assignment to the rig axis can be done automatically. If not, the assignment must be done by the operator. Anyhow, a unique name must be given.
  - Please see Appendix Axis names.
- type
  - Identifies the format of the content of the <axis> element.
  - The type attribute must be one of the following:
    - xs:integer
    - xs:float
    - xs:boolean
    - xs:double
    - xs:long
- value
  - The value of the axis. The type must match the attribute type.

<ndi_video_codec> Element

- Initial Implementation: NewTek
- Location: Metadata

Sent via NDIlib_recv_send_metadata() by the application which created the NDI receive instance.

This requests the NDI library use the specified decoding method for H.264 and H.265 NDI video streams. Note that the internal heuristics should generally be allowed to select the codec type in most circumstances.

Note this element can only be sent by an application to an NDI receiver instance, this element will never be received in a Metadata frame by an NDI sender or receiver instance.

```
<ndi_video_codec type="hardware"/>
```

<ndi_video_codec> Attributes

- type
  - Video codec type
  - Valid values are “hardware” and “software”

Proposed new metadata messages

<vancData> Element (CEA-708 & SCTE-104)

- Initial Implementation: ToolsOnAir

- Location: Video Frame Metadata

Legacy close captioning data and SCTE triggers are passed as NDI metadata by encoding the corresponding SDI vertical ancillary data packets directly as XML. Note that this method should not be used as a general solution for transitting SDI Ancillary data via NDI, but is used in this case because workflows using these protocols are very SDI centric and this method is supported by existing equipment and workflows.

This mechanism should **NOT** be used to tunnel arbitrary SDI ancillary data which can readily be represented by XML.

```
<vancData version="1.0">
  <vancPacket did="97" sdid="1" line="9">lmlZT38BWHL0/ICA...3QBWEY=</vancPacket>
  <vancPacket did="65" sdid="7" line="10">CP//AB4AAQEAAgAAAQEBAA4BAAAAAwAAAAAAAAAAA==</vancPacket>
</vancData>
```

`<vancData>` Attributes

- version
  - The `<vancData>` element must have a version attribute
  - The version attribute supported by this standard is "1.0"

`<vancData>` Children

- `vancPacket`
  - This element contains the details for one ancillary data packet
  - Any number of vancPacket elements may be contained within one vancData element

`vancPacket` Element The `vancPacket` element provides details for one ancillary data packet. The ancillary packet data is base64 encoded while the ancillary packet header details are included as attributes of the vancPacket element.

DID and SDID values and ancillary data content are per SMPTE standards ST-334 (CEA-708) and ST-2010 (SCTE-104)

The following ancillary data packet types are currently supported: * CEA-708 close caption: did="97" sdid="1" * SCTE-104 trigger: did="65" sdid="7"

```
<vancPacket did="xs:int" sdid="xs:int" line="xs:int">xs:base64Binary</vancPacket>
```

`vancPacket` Attributes

- did
  - The ancillary packet DID (Data Identifier)
- sdid
  - The ancillary packet SDID (Secondary Data Identifier)
- line
  - The line number the ancillary data packet was received on

Midi `<ndi_metadata type="midi">` Element

- Initial Implementation: Lamamix
- Location: Metadata Frame
- Initial NDI Version: 6.1

```
<ndi_metadata type="midi">
  <data>903C403D20</data>
</ndi_metadata>
```

Midi `<ndi_metadata type="midi">` Children

- data
  - Hexidecimal representation of a midi message
  - Data is read left to right (eg: 0x90 is the first byte transmitted and 0x20 is the last byte to be trasmitted)

DMX `<ndi_metadata type="dmx">` Element

- Initial Implementation: Salrayworks
- Location: Metadata Frame
- Initial NDI Version: 6.1

NOTE Some early versions of DMX used the tag `<SALRAY_DMX>` instead of `<ndi_metadata type="dmx">`. Devices receiving DMX via NDI Metadata should look for both element names. Devices sending DMX via NDI metadata should use the `<ndi_metadata type="dmx">` element with NDI SDK version 6.1 or newer.

```
<!-- Devices receiving DMX should also look for the <SALRAY_DMX> Element -->
<ndi_metadata type="dmx">
  <Universe id="1">
    <Stream>
      <Address>"1"</Address>
      <Data>01FF0304050607</Data>
    </Stream>
    <Stream>
      <Address>"8"</Address>
      <Data>1A10A0023</Data>
    </Stream>
  </Universe>
  <Universe id="2">
    <Stream>
      <Address>2</Address>
      <Data>02FF004567</Data>
    </Stream>
  </Universe>
  <Universe id="3">
    <!-- No streams for this universe -->
  </Universe>
</ndi_metadata>
```

DMX `<ndi_metadata type="dmx">` Children

- Universe
  - DMX transmit channel
  - Multiple universe elements are allowed

`Universe` Attributes

- id
  - DMX transmit channel: 1 (first channel) to the capacity of the DMX controller

`Universe` Children

- stream
  - Represents DMX data
  - Multiple stream elements are allowed

`stream` Children

- address
  - Starting DMX channel: 1 to 512
- data
  - Hexidecimal representation of DMX data
  - Data is read left to right (eg: 0x01 is the first byte transmitted for universe 1, address 1

Timed Text Captions

- Initial Implementation: Various standards bodies
- Location: Metadata Frame

Modern captioning solutions are migrating to a variety of XML based timed text formats such as:

- TTML1
- SDP-US
- IMSC1
- SMPTE-TT
- EBU-TT
- CFF-TT

As these formats are natively XML they can be easily incorporated into an NDI stream as metadata. As a transport protocol, NDI does not natively prefer any one of these standards over the others. The properly formed XML conforming to one (or more) of these standards should simply be sent as an NDI metadata frame, optionally as the child of an `<ndi_metadata_group>` element if for some reason more than one element needs to be sent in the same metadata frame.

W3C has a good overview of the various timed text caption standards including links to many of the specifications: https://www.w3.org/AudioVideo/TT/docs/TTML-Profiles.html

PTZ and Control Messages

- Initial Implementation: NewTek
- Location: Sent via SDK API calls, received as Metadata frames

`<ntk_ptz_zoom>` Element Set zoom to an absolute value: `NDIlib_recv_ptz_zoom()`

```
<ntk_ptz_zoom zoom="0.185000"/>
```

`<ntk_ptz_zoom>` Attributes

- zoom
  - Absolute value for zoom: 0.0 (zoomed in) to 1.0 (zoomed out)

`<ntk_ptz_zoom_speed>` Element Zoom at a particular speed: `NDIlib_recv_ptz_zoom_speed()`

```
<ntk_ptz_zoom_speed zoom_speed="0.005000"/>
```

`<ntk_ptz_zoom_speed>` Attributes

- zoom_speed
  - Zoom speed: -1.0 (zoom outwards) to +1.0 (zoom inwards)

`<ntk_ptz_pan_tilt>` Element Set the pan and tilt to an absolute value: `NDIlib_recv_ptz_pan_tilt()`

```
<ntk_ptz_pan_tilt pan="0.015000" tilt="-0.015000"/>
```

`<ntk_ptz_pan_tilt>` Attributes

- pan
  - Pan location: -1.0 (left) to 0.0 (centered) to +1.0 (right)
- tilt
  - Tilt location: -1.0 (bottom) to 0.0 (centered) to +1.0 (top)

`<ntk_ptz_pan_tilt_speed>` Element Pan and tilt at a particular speed: `NDIlib_recv_ptz_pan_tilt_speed()`

```
<ntk_ptz_pan_tilt_speed pan_speed="0.015000" tilt_speed="-0.015000"/>
```

<ntk_ptz_pan_tilt_speed> Attributes

- pan_speed
  - Pan speed: -1.0 (pan right) to 0.0 (stopped) to +1.0 (pan left)
- tilt_speed
  - Tilt speed: -1.0 (tilt down) to 0.0 (stopped) to +1.0 (tilt up)

<ntk_ptz_focus> Element Set focus mode and distance: * `NDIlib_recv_ptz_auto_focus()` * `NDIlib_recv_ptz_focus()` * `NDIlib_recv_ptz_focus_speed()`

```
<ntk_ptz_focus mode="auto"/>
<ntk_ptz_focus mode="manual" distance="0.485000"/>
```

<ntk_ptz_focus> Attributes

- mode
  - Sets focus mode: “manual” or “auto”
- distance
  - Focus distance: 0.0 (infinity) to 1.0 (focused as close as possible)
  - Optional, only valid when mode=“manual”

<ntk_ptz_recall_preset> Element Recall settings from a particular preset: `NDIlib_recv_ptz_recall_preset()`

```
<ntk_ptz_recall_preset index="1"/>
<ntk_ptz_recall_preset index="2" speed="0.5"/>
```

<ntk_ptz_recall_preset> Attributes

- index
  - The preset index to recall: 0 to 99
- speed
  - How fast to move to the new preset: 0.0 (slowest) to 1.0 (fastest)
  - Optional, should default to 1.0 (fastest) if not specified

<ntk_ptz_store_preset> Element Store current settings to a particular preset: `NDIlib_recv_ptz_store_preset()`

```
<ntk_ptz_store_preset index="2"/>
```

<ntk_ptz_store_preset> Attributes

- index
  - The preset index to store: 0 to 99

<ntk_ptz_white_balance> Element Sets the white balance:

- `NDIlib_recv_ptz_white_balance_auto()`
- `NDIlib_recv_ptz_white_balance_indoor()`
- `NDIlib_recv_ptz_white_balance_outdoor()`
- `NDIlib_recv_ptz_white_balance_oneshot()`
- `NDIlib_recv_ptz_white_balance_manual()`

```
<ntk_ptz_white_balance mode="auto"/>
<ntk_ptz_white_balance mode="manual" red="0.5" blue="0.5"/>
```

<ntk_ptz_white_balance> Attributes

- mode
  - White balance mode:auto, indoor, outdoor, one_shot, or manual
  - one_shot (one_push?) locks the current white balance setting
- red

  - Manual red value: 0.0 (not red) to 1.0 (very red)
  - Only present when mode="manual"
- blue
  - Manual blue value: 0.0 (not blue) to 1.0 (very blue)
  - Only present when mode="manual"

<ntk_ptz_exposure> Element Sets the exposure settings:

- `NDIlib_recv_ptz_exposure_auto()`
- `NDIlib_recv_ptz_exposure_manual()`
- `NDIlib_recv_ptz_exposure_manual_v2()`

```
<ntk_ptz_exposure mode="auto"/>
<ntk_ptz_exposure mode="manual" value="0.5"/>
<ntk_ptz_exposure mode="manual" value="0.5" gain="0.75" shutter="0.8"/>
```

<ntk_ptz_exposure> Attributes

- mode
  - Exposure mode: auto or manual
- value
  - Iris setting: 0.0 (dark) to 1.0 (light)
  - Only valid when mode="manual"
- gain
  - Gain setting: 0.0 (dark) to 1.0 (light)
  - Only valid when mode="manual"
- shutter
  - Shutter speed: 0.0 (slow) to 1.0 (fast)
  - Only valid when mode="manual"

Undocumented Mysteries

Likely deprecated element names or typos, included here for completeness.

<ntk_ptz_flip> Element

```
<ntk_ptz_flip enabled="true">
```

Referenced in `NDIlib_Send_VirtualPTZ.cpp` but does not appear anywhere in the NDI SDK source code

<ntk_ptz_white_balance mode="one_push"/> White Balance

Referenced in `NDIlib_Send_VirtualPTZ.cpp` but does not appear anywhere in the NDI SDK source code which uses "one_shot" instead.